

55 JAIIO, ASAlD 2026

Not All Instructions Are Forgotten Equal

(No todas las instrucciones se olvidan igual)

Tomás Korenblit

Licenciatura en Ciencia de Datos, UNSAM

github.com/korentomas/not-all-instructions

EL PROBLEMA

Agentes LLM

Los agentes corren cientos de turnos llamando herramientas sin que nadie revise cada paso, mientras que sus límites operativos y salvaguardas se declaran al principio de la sesión.

Cada instrucción queda enterrada bajo el output que se acumula después de ella y compite por atención contra todo lo que sigue.

Las instrucciones se dan una sola vez y tienen que valer durante toda la corrida.

EL PROBLEMA

Context rot

La adherencia se degrada a medida que el contexto crece, con caídas promedio documentadas del 39% en configuraciones multi-turno (Laban et al., 2025), porque la atención no se distribuye uniforme sobre la ventana.

Los estudios existentes reportan adherencia promedio sobre conjuntos de instrucciones.

El promedio no dice cuál instrucción cayó.

ESTE TRABAJO

Retención por instrucción

Medimos qué retiene el modelo instrucción por instrucción, y la retención resulta muy despareja.

-2,4 a +5,4

rango del efecto de repetir,
en log-odds

3 de 12

instrucciones se benefician
de ser repetidas

8 de 12

ya se cumplen por defecto

MÉTODO

Protocolo

Sesiones de código simuladas de 25 turnos, con 12 preferencias plantadas como pedidos casuales del usuario.

turnos 0 a 19, se inyectan archivos del codebase (98K a 203K tokens)

20 a 25, prueba

▲ 6 tempranas (turnos 1 a 7)

▲ 6 tardías (14 a 19)

El código generado en los turnos de prueba se puntúa con checkers determinísticos en una escala ordinal de 0 a 3. El baseline corre las mismas sesiones sin plantar nada.

MÉTODO

Anatomía de una conversación

TURNOS 0 A 2

archivos del codebase inyectados en el contexto

TURNO 3, USUARIO

"Important: when writing array operations for this project, always use NumPy broadcasting instead of for loops."

TURNOS 4 A 19

más archivos y tareas... el contexto crece hasta 98K a 203K tokens

TURNO 22, PRUEBA

"Escribí las operaciones de array para normalizar estas columnas."

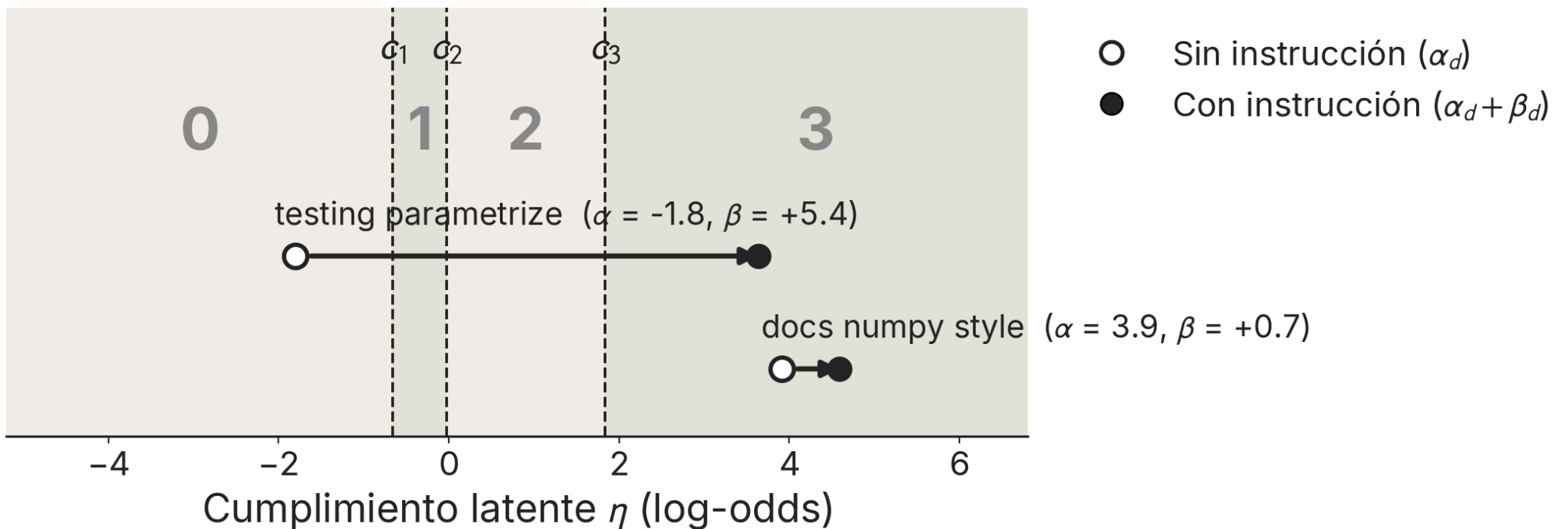
RESPUESTA DEL MODELO

código generado, que un checker determinístico puntúa de 0 a 3

Entre el pedido y la evaluación hay decenas de miles de tokens.

El modelo

El puntaje 0 a 3 es una posición sobre una escala latente dividida por tres cortes, y decir la instrucción desplaza esa posición en β_d .



Un efecto grande importa solo si arrancás abajo.

MÉTODO

El modelo

Una regresión logística ordenada con efectos jerárquicos estima un efecto por instrucción.

$$score_i \sim \text{OrderedLogistic}(\eta_i, c)$$

el puntaje 0 a 3 es ordinal y cae según dónde queda η respecto de tres cortes ordenados $c_1 < c_2 < c_3$

$$\eta_i = \alpha_{d[i]} + \beta_{d[i]} \cdot told_i$$

α_d es lo que el modelo hace por defecto para la decisión d , mientras que β_d es cuánto cambia al decirla

$$\alpha_d \sim N(\mu_\alpha, \sigma_\alpha), \beta_d \sim N(\mu_\beta, \sigma_\beta)$$

las 12 instrucciones comparten información mediante partial pooling, que protege contra los pocos datos por celda

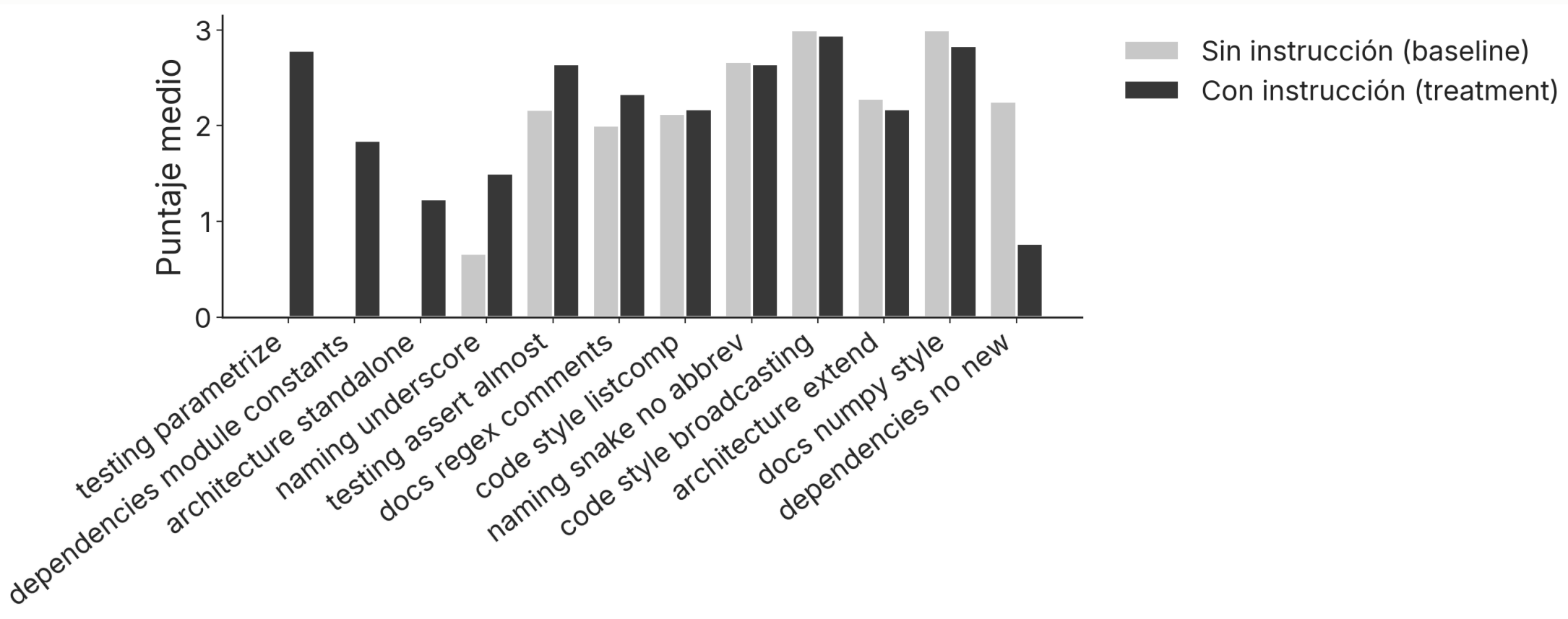
$$\mu_\beta \sim N(0, 1), \sigma_\beta \sim \text{Exp}(1)$$

priors débiles centrados en cero, sin dirección asumida para el efecto

244 observaciones, 28 conversaciones, 5 modelos, 3 codebases. σ_β mide cuánto varían las instrucciones entre sí, y su posterior da 2,1.

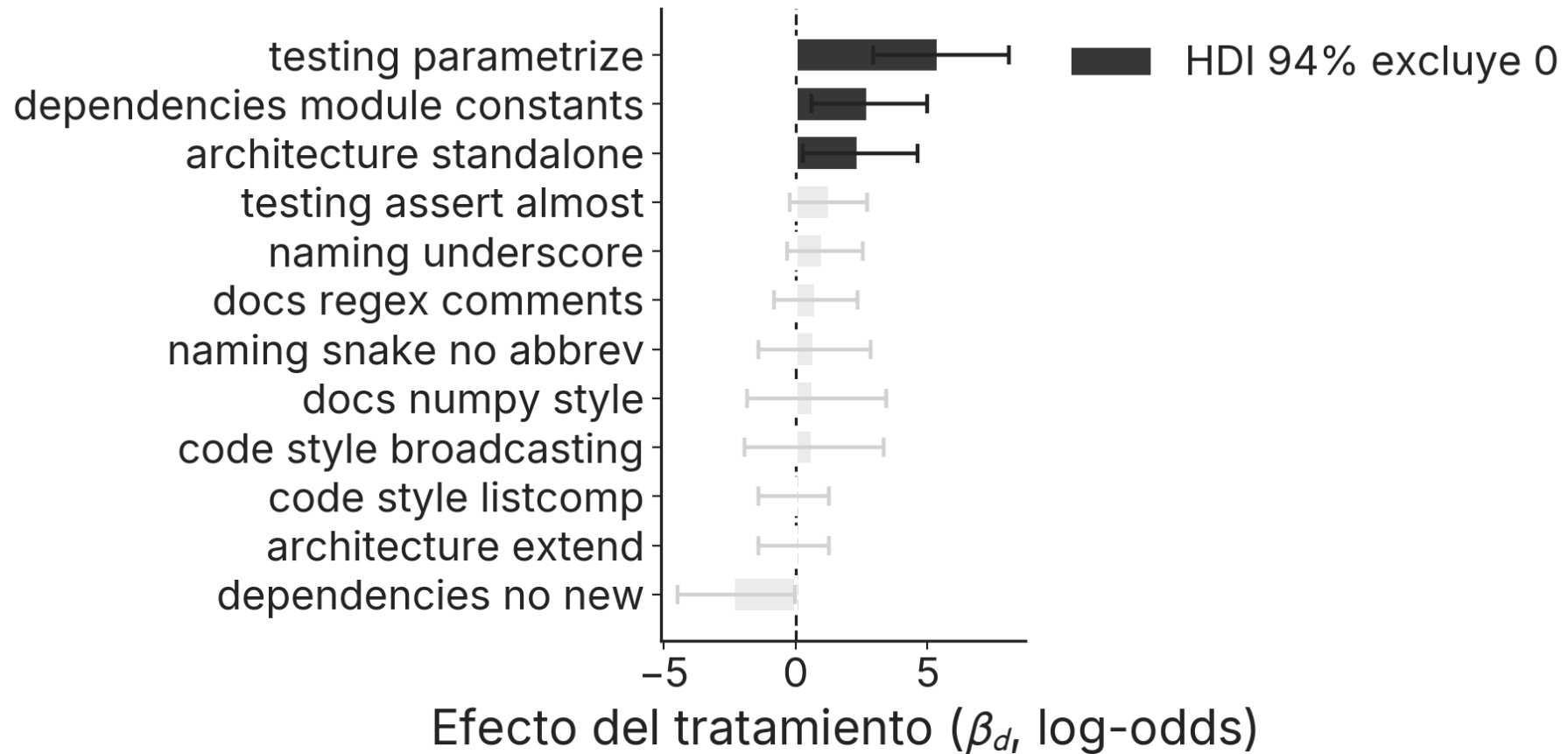
RESULTADOS

Datos crudos



Tres patrones visibles antes de cualquier modelo.

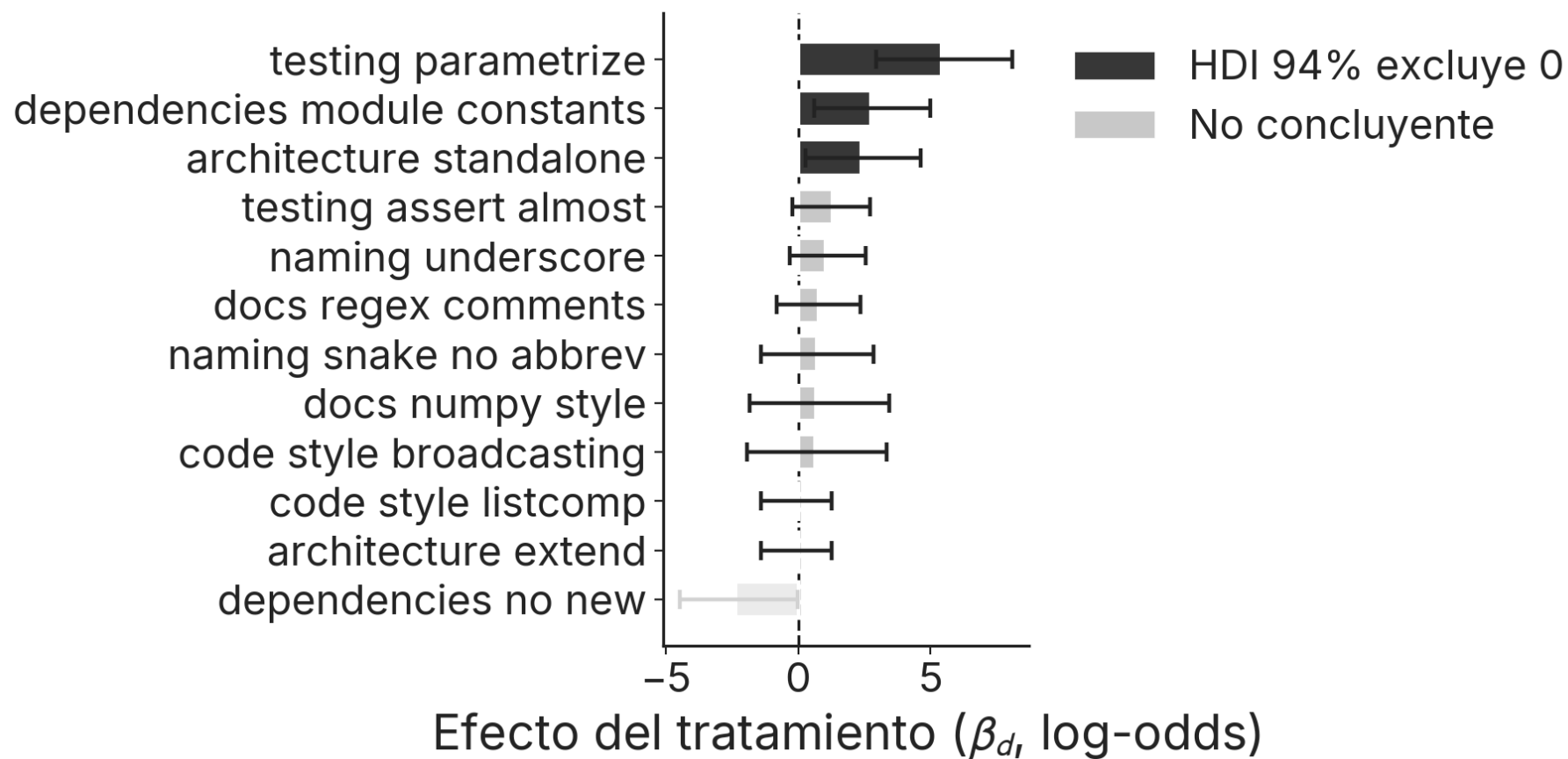
Efectos por instrucción



Tres instrucciones se benefician de ser repetidas.

testing parametrize pasa de no usarse nunca (baseline 0,00) a retenerse cuando se pide ($\beta = 5,4$)

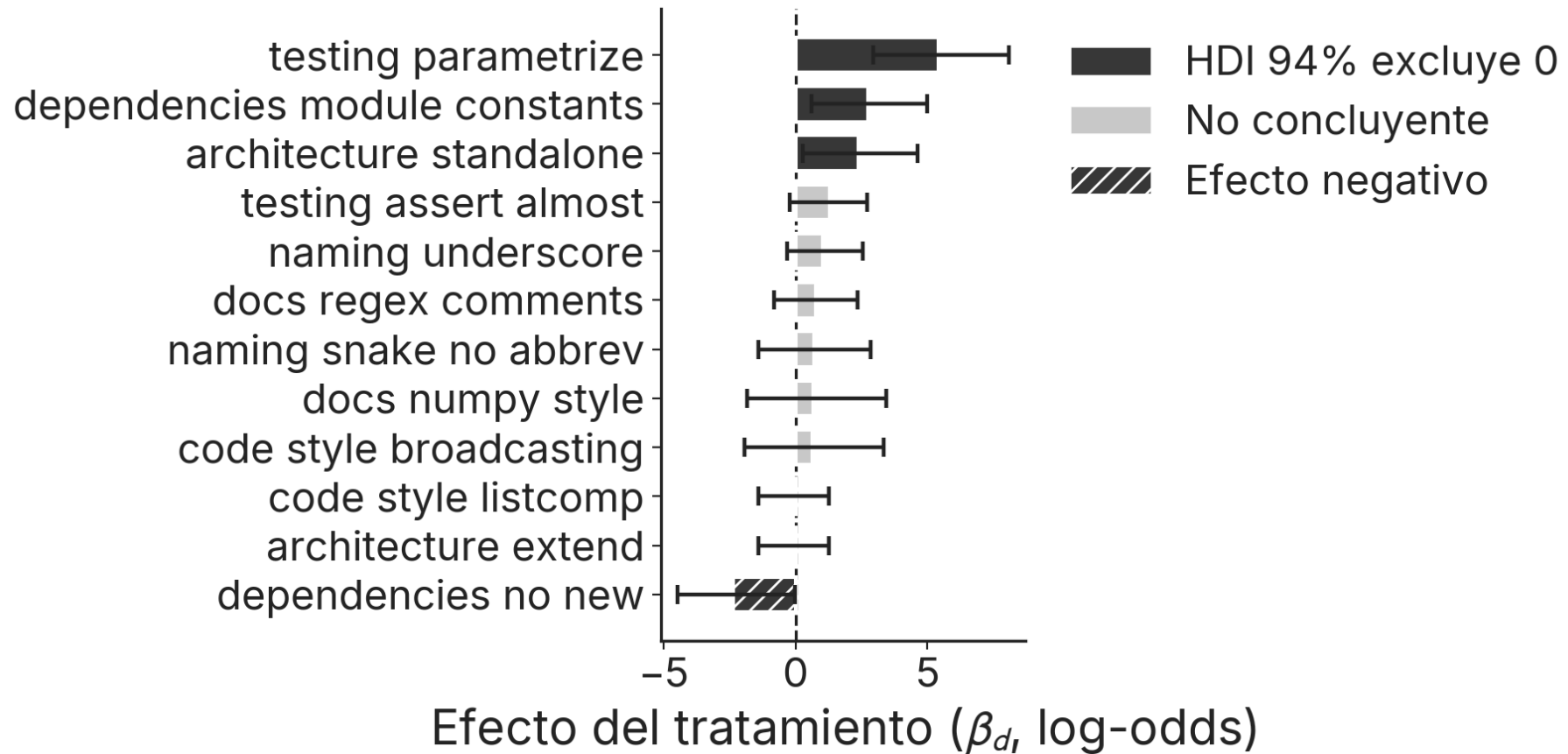
Efectos por instrucción



Ocho ya se cumplen por defecto, así que repetirlas no cambia nada.

RESULTADOS

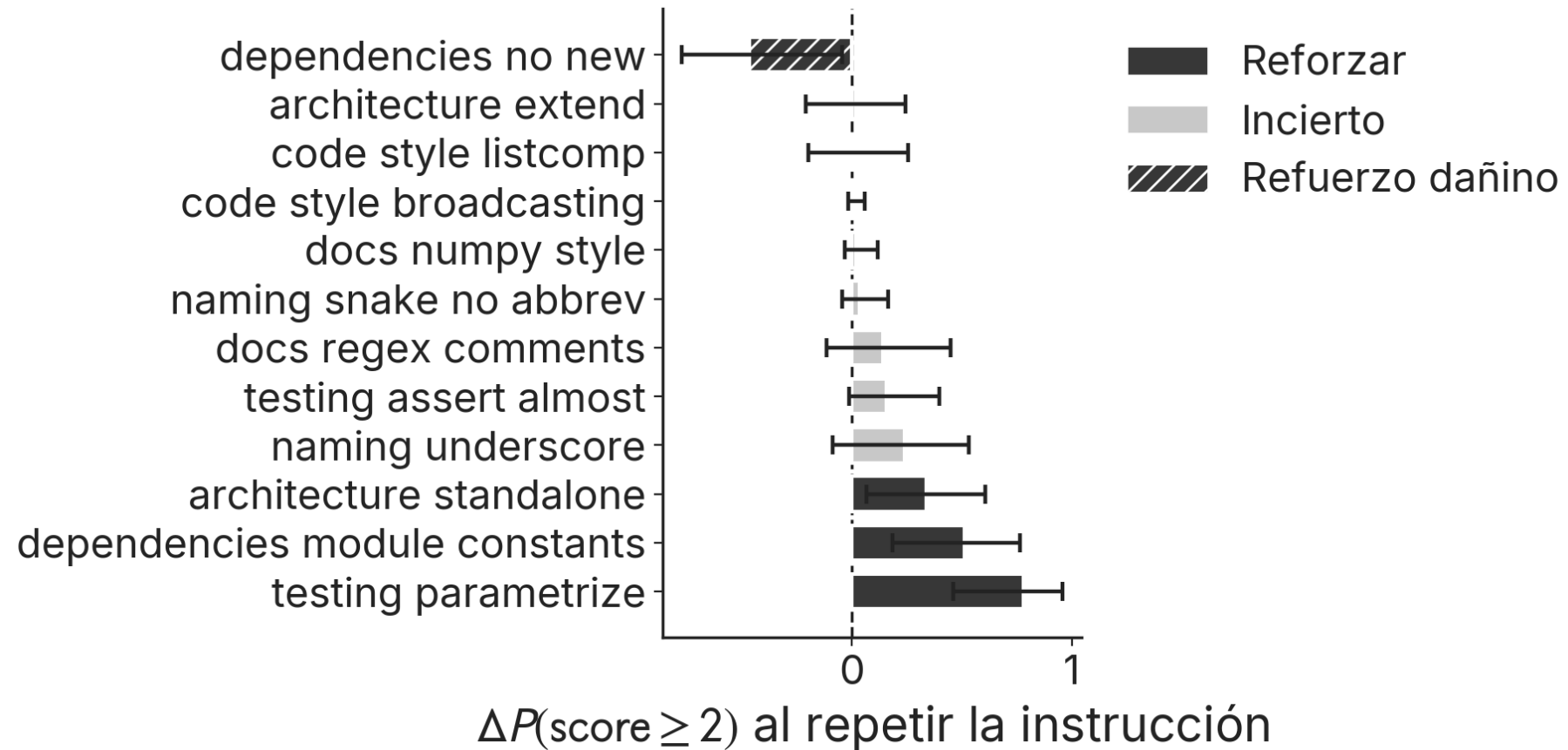
Efectos por instrucción



Una empeora al repetirla, aunque sospechamos un artefacto de medición.

el checker de dependencies no new cuenta como nuevo cualquier import, incluidos los ya presentes en el archivo

Política de refuerzo



Repetir todas las instrucciones desperdicia tokens en 9 de 12.

una ganancia baja no justifica descartar una regla de seguridad, solo ahorra repetición en preferencias ordinarias

RESULTADOS

Modelos y codebases

Los interceptos por modelo y por codebase concentran su posterior cerca de cero ($\sigma \approx 0,3$), de modo que la heterogeneidad vive en las instrucciones y no en quién ejecuta ni dónde.

La retención tampoco sube de 26B a 120B parámetros.

La desigualdad no es capacidad disfrazada.

Con baseline de un solo modelo esto sostiene invarianza débil, no ausencia de efectos por modelo.

DISCUSIÓN

Salvaguardas

Una preferencia de código y un guardrail son el mismo objeto, a saber una instrucción dicha una vez que compite por atención, por lo que la retención medida acá es un proxy de cómo aguantaría una salvaguarda declarada.

Como la seguridad es conjuntiva, el cumplimiento agregado puede verse alto mientras la única regla que importa ya cayó.

Un agente desplegado necesita monitoreo por instrucción.

Conclusiones

- La heterogeneidad entre instrucciones es grande, con efectos de $-2,4$ a $+5,4$ log-odds y $\sigma_{\beta} = 2,1$.
- Solo 3 de 12 instrucciones pagan el refuerzo, mientras que una política uniforme gasta en las 12.
- El marco per-instrucción habilita monitoreo selectivo, con BKT como dirección natural.

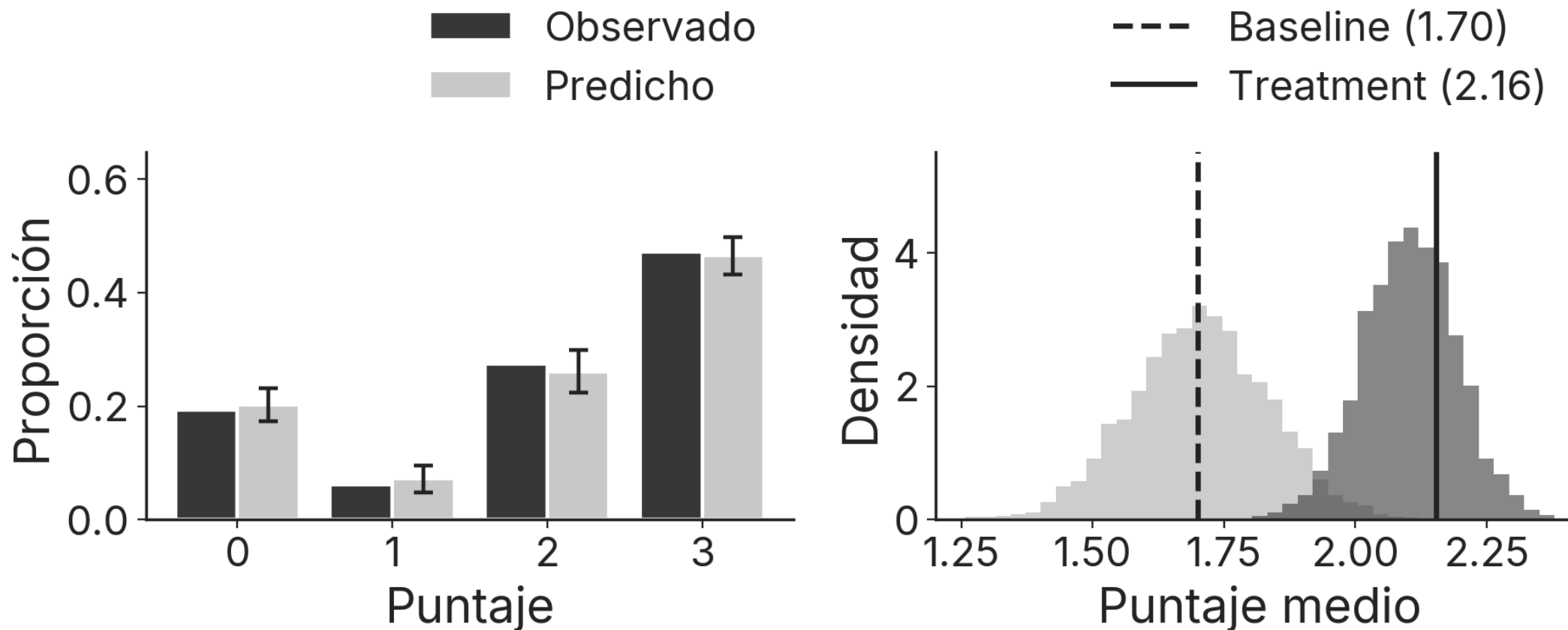
La retención es medible, despareja y accionable.

`github.com/korentomas/not-all-instructions`

Gracias.

Backup

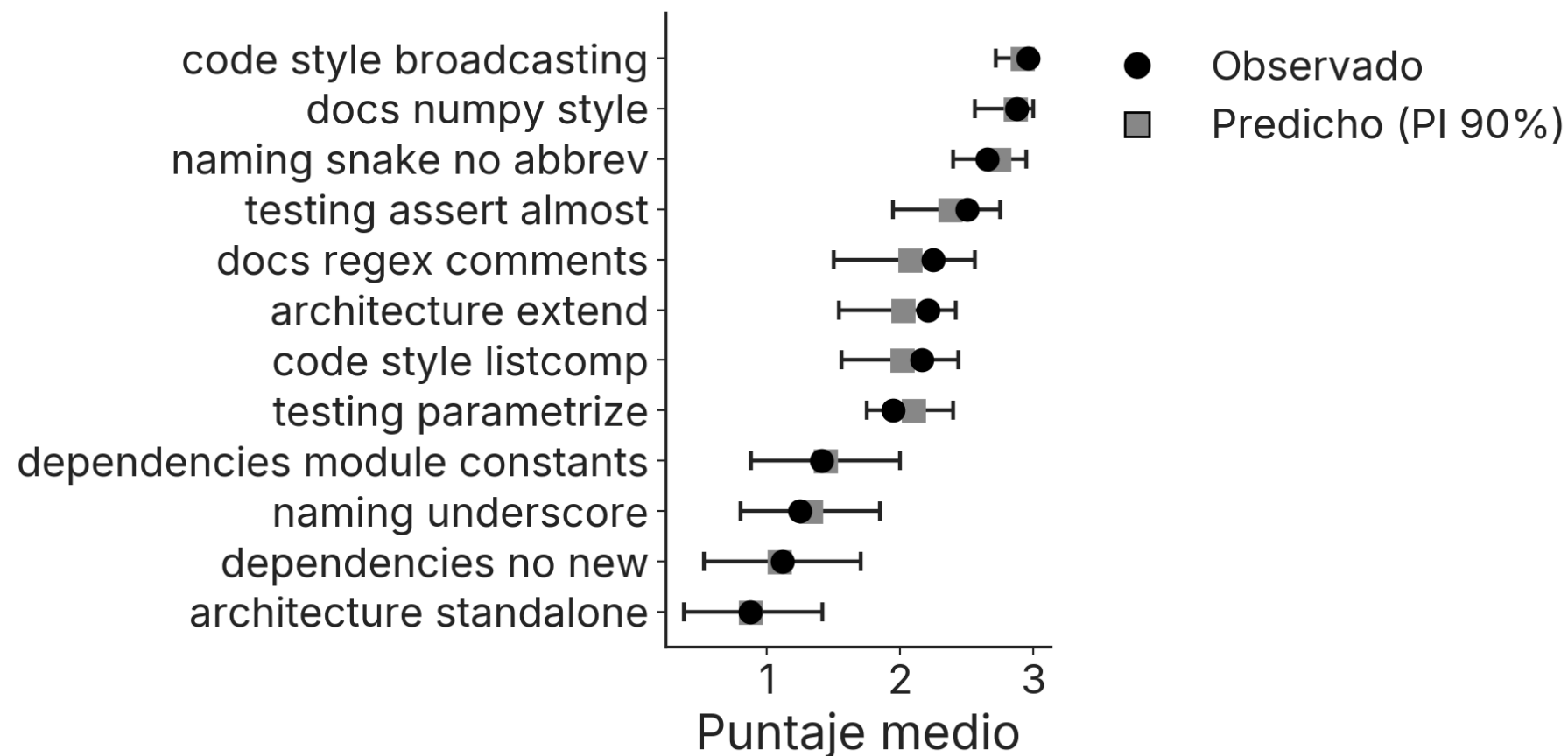
Posterior predictive check



El modelo reproduce la distribución observada.

cero divergencias, $\hat{R} \leq 1,01$, ESS mínimo 1311, medias 1,70 (baseline) y 2,16 (treatment)

PPC por decisión



Las 12 medias observadas caen dentro del intervalo predictivo del 90%.

BACKUP, ESPECIFICACIÓN

Especificación

Ordered logit con cutpoints ordenados por incrementos positivos, efectos jerárquicos no centrados por tipo de decisión, y priors débilmente informativos en escala log-odds, a saber $\mu_\alpha \sim N(0, 1.5)$, $\mu_\beta \sim N(0, 1)$ centrado en cero, $\sigma \sim \text{Exp}(1)$.

PyMC con sampler nutpie (NUTS), 4 cadenas de 1000 draws tras 1000 de tuning. Prior y posterior predictive checks en el repo.

BACKUP, EL EFECTO NEGATIVO

El artefacto

El checker cuenta cualquier sentencia import como nueva, incluidos los imports ya presentes que el modelo vuelve a mostrar al reescribir un archivo, de modo que el puntaje castiga una conducta que la instrucción no prohíbe.

La dirección negativa es creíble ($P(\beta < 0) = 0,98$) pero la magnitud queda abierta, y separar la negación del contenido (comparar "avoid new imports" con "use only existing imports") lo resolvería.

BACKUP, TRABAJO POSTERIOR

Trabajo posterior

- Omission vs commission (arXiv 2604.20911) encuentra que las prohibiciones decaen del 73% al 33% mientras que los requerimientos se mantienen, con señales de auditoría que se ven sanas mientras la restricción ya falló.
- Governance Decay (2606.22528) muestra salvaguardas borradas silenciosamente al compactar contexto.

Confirma la dirección, aunque mide a nivel de categoría y no de instrucción individual.

BACKUP, ROBUSTEZ

Robustez

Bajo un refit solo-Qwen (baseline y treatment del mismo modelo) dos de los tres efectos positivos retienen su HDI sobre cero, y el ranking de refuerzo se mantiene estable.

Sensibilidad a priors y exclusión del sentinel -1 documentadas en el repo.